

10

Cyber Security
Codec Technologies India
Internship 2026

Low-Interaction Honeypot

Fake SSH/HTTP/Telnet services
+ threat analytics

AT A GLANCE

25

Sessions

24

Cred attempts

3

Services

0

Auth granted

root / admin (8)

Top user

7 / 7

Tests re-run

127.0.0.1

Bind

VERDICT

TESTS 7/7

RE-RUN HERE

SYNTHETIC

DEFENSIVE ONLY

LOCALHOST ONLY

Objective & brief mapping

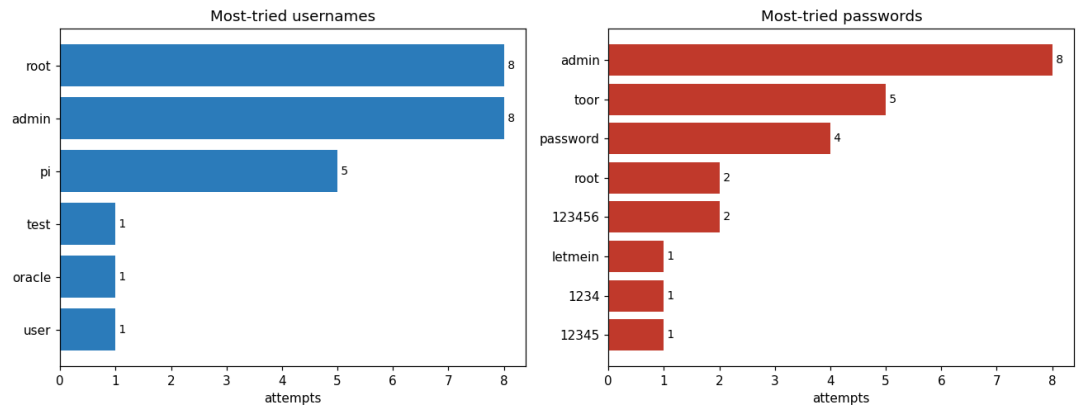
The brief asked for a honeypot with threat logging. I built a low-interaction honeypot: three fake services present convincing banners/prompts, record everything the client sends (source, timestamps, bytes, credentials, transcript), then deny the attempt. A bundled attack simulator plays a credential-spraying bot so it all runs on one machine without ever exposing anything.

Architecture

server.py runs SSH (2222), HTTP (8080) and Telnet (2323), each on its own thread with a per-session handler that presents a service and captures input but never grants access. A per-IP RateLimiter, a thread-safe EventLog, then analytics that render two PNG dashboards and a self-contained HTML dashboard.

```
simulator -> SSH:2222 HTTP:8080 Telnet:2323 (127.0.0.1)
handler: banner -> prompt -> DENY ; RateLimiter ; EventLog
-> events.json -> aggregate -> summary.json + PNGs + dashboard.html
```

Honeypot — captured credential attempts



Most-tried credentials (results/top_credentials.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # start -> simulate -> stop -> analyze
python3 unittest-style runner over tests/ # 7 tests
```

The artefacts each figure and table above are drawn from:

- **summary.json** — session/credential stats
- **events.json** — one JSON event per session
- **honeypot.log** — human-readable log
- **top_credentials.png / hits_over_time.png** — dashboards
- **dashboard.html** — self-contained HTML dashboard

Method & implementation

Every session becomes one structured event (UTC connect/disconnect, src_ip:port, service, bytes_sent, credentials, redacted-safe transcript). The simulator submits fake logins from a small weak-credential wordlist, deterministically seeded so runs reproduce. Rate-limited excess connections are still logged (as rate_limited) but answered with one line and closed.

Results

From results/summary.json: one full run logged 25 sessions and captured 24 credential attempts across 3 services, granting 0 authentications.

Metric	Value
Sessions / cred attempts	25 / 24
Per service	ssh 8, http 9, telnet 8
Bytes served / auth granted	3111 / 0
Top usernames	root (8), admin (8), pi (5)
Top passwords	admin (8), toor (5), password (4)

Fake services captured

Service	Port	Captured
SSH	2222	user/pass + transcript
HTTP	8080	form user/pass
Telnet	2323	user/pass
(all)	--	denied, 0 auth granted

Verification

KEY TAKEAWAY Three fake services capture credentials and grant access to nothing; a bundled bot drove 25 sessions and 24 captured logins, all analysed into a dashboard.

10

Cyber Security
Codec Technologies India
Internship 2026

Low-Interaction Honeypot

Fake SSH/HTTP/Telnet services
+ threat analytics

AT A GLANCE

25

Sessions

24

Cred attempts

3

Services

0

Auth granted

root / admin (8)

Top user

7 / 7

Tests re-run

127.0.0.1

Bind

VERDICT

TESTS 7/7

RE-RUN HERE

SYNTHETIC

DEFENSIVE ONLY

LOCALHOST ONLY

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Re-ran here, 7/7 (~5 s). Listeners bind to 127.0.0.1 and the simulator drives them in-process.

Test group	Checks	Result
Capture	connection + creds logged, HTTP form captured	PASS
Safety	never grants auth	PASS
Rate limit	excess connections marked	PASS
Analytics	aggregation arithmetic + dashboard	PASS
Total	7 tests	7/7

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

A low-interaction sensor; higher-interaction tooling is referenced but deliberately not deployed.

Ran here	Real tool	Note
3 fake listeners	Cowrie (SSH/Telnet)	emulated shell = higher risk
local simulator	real internet exposure	out of scope (legal/privacy)
per-IP RateLimiter	fail2ban / iptables	persisted in production

Ethics, data integrity & declared substitutions

Strictly defensive. Everything is synthetic and self-contained: listeners bind to localhost, the "attackers" are the bundled simulator, and captured credentials are its own fake values from a public weak-password list. **Declared scope:** deploying a honeypot on the public internet carries legal/privacy/operational consequences and is explicitly out of scope; the credential-capture code must not be repurposed against systems you are not authorised to run it on.

Learnings

Designing services that capture but never authenticate reframed logging for me: in a honeypot the captured credential *is* the intelligence, so the transcript redaction and the event schema matter as much as the listener. On a real sensor the same aggregation that looks trivial on 127.0.0.1 is what surfaces botnet spread.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Low-interaction only: banners + one login exchange, no shell/filesystem emulation.
- The SSH listener speaks a simplified cleartext prompt, not the real binary protocol.
- Source-IP analytics is trivial locally (everything is 127.0.0.1).
- Rate-limiting is per-IP in-memory and resets on restart.
- Everything binds to 127.0.0.1 and grants access to nothing.
- Captured credentials are the simulator's own fake values from a public weak-password list.
- One run: 25 sessions, 24 credential attempts, 0 authentications granted.
- Deploying a honeypot on the public internet is explicitly out of scope.
- One run: 25 sessions, 24 credential attempts, 3 services.
- 3111 bytes of banners served; 0 authentications granted.
- Top usernames: root (8), admin (8), pi (5).
- Top passwords: admin (8), toor (5), password (4).
- Per service: ssh 8, http 9, telnet 8.
- dashboard.html fetches events.json and renders KPI cards.
- RateLimiter is a per-IP sliding window.
- References: Cowrie, The Honeynet Project, T-Pot.
- Reference: Cowrie; The Honeynet Project; T-Pot platform.
- Reference: MITRE ATT&CK; Valid Accounts (T1078).
- Reference: SANS Honeypots reading; OWASP Credential Stuffing.
- summary.json records 25 sessions and 24 credential attempts.